Brünel ERP & Field Operations Platform — Master Architecture & Implementation Plan

1. Current Workspace vs. GitHub Repository Comparison

Workspace Overview

Your workspace contains two main applications:

1. : The public corporate website (Next.js 16 / React 19 / Three.js). **Per your instruction, we will not touch or modify this directory.**
2. : The internal Brünel OS application linked to `https://github.com/bruenel/app.bruenel.de`.

Comparison with `github.com/bruenel/app.bruenel.de` (`main` branch)

- **Git Status:** Local `HEAD` (`ccfd369`) matches `origin/main` (`ccfd369`).
- **Local Uncommitted Changes:**
  - : Added `engine = None` fallback if database engine creation fails.
  - : Minor dependency version bumps.
  - : Untracked specification document (460 lines).
- **Existing Stack & Modules in `app.bruenel.de`:**
  - **Backend:** Python FastAPI (), SQLAlchemy 2.0 (), JWT Auth (`python-jose` + `bcrypt`). Configured for **Neon Serverless PostgreSQL** (in `.env`) with local SQLite (`bruenel_os.db`) / Docker Postgres fallback.
  - **Frontend:** React 18 + Vite + React Router v6 + Lucide Icons (— currently a single 1,482-line file).
  - **Current Existing Features:**
    1. **Auth & User Management** () — `Owner`, `Partner`, `Mitarbeiter` roles + `allowed_kst`.
    2. **Supplier Matrix** () — KST-based supplier pipeline.
    3. **Internal Mail & Signature Generator** ().

4. **Legal Vault** () — currently writes to local `/tmp/secure_vault` (ephemeral on Vercel).
5. **BI Dashboard & Cookie Tracking** ().

2. Multi-Domain & Portal Architecture (`app.bruenel.de` vs `portal.bruenel.de`)

How `app.bruenel.de` and `portal.bruenel.de` Connect

We can serve **both portals from the unified frontend codebase** (or workspace monorepo) pointing to the same FastAPI backend:

- **`app.bruenel.de` (Management & ERP Portal):**
  - Accessible **only** to accounts with management/representative authorization (`Tier 1–4` or custom module grants).
  - Displays all business & project management modules in a desktop-first (responsive) command center, dynamically filtered by the user's **permitted modules** (`allowed_modules`) and **Row-Level Security (RLS)** scope.
  - If a regular Tier 5 Field Employee attempts to log into `app.bruenel.de`, they are automatically denied or redirected to `portal.bruenel.de`.
- **`portal.bruenel.de` (Employee Self-Service & Field Operations PWA):**
  - Mobile-first Progressive Web App (PWA) designed for all employees in the field and office.
  - **Employee Features:**
    - **My Tasks & Assignments:** Daily assigned project, region (PLZ), building objects, and instructions.
    - **Smart Check-In / Check-Out (Working Hours):** Log daily hours, select vehicle, tools, and team members; confirm tool return and material consumption (`Buchen` / `Schwund`) at check-out.
    - **Holiday Requests (`Urlaub`):** Submit Daily, Half-day, or Long-term leave requests and track approval status.
    - **Sickness Notification (`Krankmeldung`):** Report sickness absence immediately with date range and upload doctor's note (`AU-Bescheinigung` photo/PDF).
    - **My Salary Documents (`Lohnabrechnung`) & HR Docs:** Secure monthly payslip viewer/downloader, tax documents, and employment contracts.

        - **Expenses & Receipts:** Upload fuel/material receipts with photo capture.
        - **Field Sales & Upselling:** On-site hardware sales (CASH/CARD), automatic vehicle stock check, and live personal commission (`Provision`) tracker.
- **Localhost Development:**
    - On localhost, we can switch between Management View (`app.bruenel.de`) and Employee Portal (`portal.bruenel.de`) via URL query/path (e.g., `localhost:5173/` vs `localhost:5173/portal` or subdomain `portal.localhost:5173`) while in production hostname detection (`window.location.hostname === 'portal.bruenel.de'`) locks the interface to the respective portal automatically.

3. User Layers, RBAC Tiers & Granular Module Access Control

Per your requirement (*"each account must be given access to access each module"*), we will combine the **5-Tier RBAC + Delegation (`Vertretung`)** from your blueprint with a **Granular Module Permission Matrix** per user account.

3.1 The 5 Role Tiers + Legacy Compatibility

We will unify the existing `users` table with the new `employees` schema so existing accounts (`Owner` , `Partner` , `Mitarbeiter`) continue working seamlessly while upgrading to the 5-Tier architecture:

Tier	Role Identifier	Portal Access	Default Authority & Scope
Tier 1	`SUPER_ADMIN` (`Owner`)	`app.bruenel.de` + `portal.bruenel.de`	Full CRUD across all modules, user/module permission management, dynamic JSONB schema attributes, accounting, BI, Vault.
Tier 2	`PROJECT_MANAGER` (`Projektleiter`)	`app.bruenel.de` + `portal.bruenel.de`	Full read/write for assigned Projects, Objects, Regions, Team scheduling, Lager/Vehicle inventory, Leave/Sickness approvals for assigned teams. Supports **Delegation (`Vertretung`)**.
Tier 3	`HR_FINANCE` (`HR & Finance Admin`)	`app.bruenel.de` + `portal.bruenel.de`	Full access to Employee profiles, working hours reports, Sickness/Holiday records, monthly Payslip (`Lohnabrechnung`) uploads, Expense approvals, Accounting & Commission exports.
Tier 4	`TEAM_LEAD` (`Vorarbeiter` / Representative)	`app.bruenel.de` (scoped) + `portal.bruenel.de`	Can access authorized modules on `app.bruenel.de` (e.g., Project Objects, Inventory Buchen, Team Check-in/out, Equipment handover) restricted to their assigned teams/projects.
Tier 5	`FIELD_EMPLOYEE` (`Mitarbeiter` / Technician)	`portal.bruenel.de` ONLY (unless explicitly granted a module on `app`)	Strictly scoped via PostgreSQL RLS (`employee_id`) to their own tasks, working hours, leave/sickness requests, personal payslips, expenses, and field sales.

3.2 Per-Account Module Access Matrix (`user_module_permissions`)

In User Management on `app.bruenel.de` , Super Admins can toggle **per-account module permissions** (`can_view` , `can_create` , `can_edit` , `can_delete` , `can_approve`) for every module:

- `mod_hr` (HR, Employees, Working Hours, Leave & Sickness, Payslip Distribution)
- `mod_projects` (Projects, Regions, Building Objects, Daily Task Assignments)
- `mod_inventory` (Lagers, Materials, Event-Sourced Stock Ledger, Schwund Reconciliation)
- `mod_fleet` (Vehicles & High-Value Equipment / Tool Tracking)
- `mod_calendar` (Centralized Master Interactive Calendar)
- `mod_accounting` (Expenses, Invoices, Financial Reports, DATEV/CSV Exports)
- `mod_sales` (Field Sales Catalog, Transactions, Commissions Dashboard)
- `mod_suppliers` (Existing Supplier Matrix)
- `mod_vault` (Existing Legal Vault)
- `mod_bi` (Existing BI & Website Analytics)

- `mod_mail` (Existing Internal Mail & Signatures)

4. What Changes We Need on the Current App

1. **Frontend Refactoring (`/frontend/src`):**
    - Break up the monolithic 1,482-line into a clean modular structure:
        - `/src/core/` (Auth context with Refresh Token support, API client, Role & Module Route Guards, Theme manager)
        - `/src/portals/management/` (`app.bruenel.de` layout & management modules)
        - `/src/portals/employee/` (`portal.bruenel.de` mobile-first PWA layout & employee self-service views)
        - `/src/modules/` (Isolated UI modules for HR, Projects, Inventory, Fleet, Calendar, Accounting, Field Sales, plus existing Supplier, Mail, Vault, BI).
2. **Backend Restructuring (`/backend`):**
    - Create `/backend/core/` per Chapter 1.3 of the blueprint:
        - `security.py` : 15-min Access JWT + 7-day HTTP-Only Refresh Cookie, `bcrypt` (cost 12), RBAC & Module Permission dependency guards.
        - `rls.py` : SQLAlchemy session middleware that sets `SET LOCAL app.current_employee_id = '...'` and `SET LOCAL app.current_role = '...'` on PostgreSQL transactions so Row-Level Security policies are enforced automatically.
        - `audit.py` : PostgreSQL trigger & application-level immutable `audit_logs` handler.
        - `storage.py` : Unified Pre-signed URL storage service for photos, videos, and PDFs.
    - Implement versioned REST endpoints under `/api/v1/*` while keeping `/api/*` aliases for existing features so nothing breaks.
3. **Database Upgrade:**
    - Upgrade from basic SQLAlchemy table creation to full **PostgreSQL Schema Initialization & Migration** supporting UUID primary keys, ENUM types, `JSONB` dynamic attributes, `equipment_exclusive_assignment` constraints, `trigger_field_sale_inventory` , `audit_logs` triggers, `delegation` (`Vertretung`), and RLS policies.
    - Add missing fields for your employee portal requirements:

- **Sickness Records (`sickness_logs` or extended `leave_requests`):** Support `Sick` / `Krankmeldung` with `au_document_id` (doctor's certificate), notification timestamp, and HR status.
- **Employee Tasks (`tasks` / enhanced `assignments`):** Support detailed task descriptions, priority, object link, checklist items, and completion media (photos/videos).
- **Monthly Payslips (`payslips` / `documents`):** Add `month`, `year`, `net_amount`, `commission_included` metadata linked to `employee_id` so employees can browse their salary documents by month/year in `portal.bruenel.de`.
- **Accounting Expenses (`expenses`):** Track employee expense submissions, category, net/VAT/gross amounts, receipt media link, and approval status (ready for manual review now, and n8n webhook automation later).

5. Database & File / Video Storage Solutions

5.1 Database Solution

- **Production & Cloud Dev:** You already have a **Neon Serverless PostgreSQL** instance configured in `backend/.env` (`ep-falling-cloud-alqmwfqm-pooler.c-3.eu-central-1.aws.neon.tech`). Neon supports PostgreSQL 15/16, `uuid-ossp`, `pgcrypto`, `JSONB`, Triggers, and Row-Level Security (RLS).
- **Local Offline Development:** We will use **PostgreSQL 15** (via your `docker-compose.yml` or local Postgres service) so that all PostgreSQL-specific features (UUIDs, JSONB, RLS policies, PL/pgSQL triggers) run 100% identically offline on `localhost` before we push to production.
- **Driver Upgrade:** Replace `pg8000` with `psycopg[binary]` (or `psycopg2-binary`) in `requirements.txt` for full native support of PostgreSQL JSONB, UUIDs, SSL, and session-level RLS settings.

5.2 Solutions for Uploading Files, Photos & Large Videos

Because your backend deploys to Vercel/containerized environments (and browsers uploading 50MB–500MB site inspection videos through a Python API will hit timeout/payload limits and throttle the server), **Direct-to-Object-Storage via Pre-Signed URLs** is the industry best practice:

> TIP
>
> **Recommended Storage Architecture (3-Tier Options)**

Option	Technology	Why It Fits Brünel ERP Best	Cost / Setup
Option A (Recommended for Production)	**Cloudflare R2** (S3-Compatible)	**Zero egress (download/streaming) fees—**critical when managers stream site videos or employees download PDFs. Fully S3-compatible (`boto3`), EU jurisdiction lock (GDPR/DSGVO compliant), supports multipart chunked video uploads up to 5GB+ via pre-signed URLs.	10 GB free/mo, then $0.015/GB storage, $0 bandwidth.
Option B (100% Self-Hosted / Local Offline)	**MinIO** (in `docker-compose.yml`) + Local Disk fallback	Runs locally in Docker on `localhost:9000` using the exact same `boto3` S3 code as Cloudflare R2 / AWS S3. Zero cloud dependency during offline development.	Free (Open Source).
Option C (Enterprise Cloud)	**AWS S3 (`eu-central-1` Frankfurt)** + **Google Drive API** (for final HR/Accounting archives)	Matches Chapter 2.1 of your blueprint: S3 handles high-frequency site photos, videos, and receipts; Google Drive Service Account stores final compiled PDFs and `Lohnabrechnung` archives.	Standard AWS S3 storage + egress pricing.

How Video & File Uploads Will Work in Our Code:

1. **Unified Storage Adapter (`/backend/core/storage.py`):**
    - Supports **Local/MinIO mode** (for offline localhost development) and **S3/Cloudflare R2 mode** (for production) via `.env` switch.
2. **Pre-Signed Direct Upload Flow (for Videos & High-Res Photos):**
    - Frontend requests `POST /api/v1/media/presign` (sending filename, MIME type, module reference: `ObjectMedia` , `Expense` , `Lohnabrechnung` , `Krankmeldung`).
    - Backend validates RBAC permissions, creates a `documents` record in PostgreSQL, and returns a time-limited **Pre-Signed PUT URL**.
    - Frontend uploads the video/file **directly to S3/R2/MinIO** with a live progress bar (and optional client-side image compression for site photos).
    - For viewing, the backend generates short-lived **Pre-Signed GET URLs** (with HTTP Range header support so videos stream smoothly on mobile and desktop without downloading the entire file first).

6. Accounting Module & Deferred n8n Automation Documentation

Per your instructions:

1. **We WILL build Module 6 (Accounting, Expenses, Receipts, & Payroll/Payslip Management) directly in the app now:**
    - Employees can upload expense receipts and view monthly payslips (`Lohnabrechnung`) in `portal.bruenel.de` .
    - HR/Finance Admins & Owners can review/approve expenses, upload/assign monthly payslips to employees, track project costs, and export accounting ledgers (CSV/DATEV format) on `app.bruenel.de` .
2. **We WILL SKIP building the live n8n workflows & Telegram bots for now.**
3. **We WILL create a dedicated `N8N_AUTOMATION_GUIDE.md` instruction blueprint** documenting:
    - All webhook payloads, API service account endpoints, and authentication headers exposed by our backend.
    - Step-by-step instructions to connect n8n later for:
        - Automated PDF compilation of approved expenses to Google Drive.
        - Automated `Lohnabrechnung` (payslip) ingestion from Google Drive / DATEV and auto-matching to `employee_number` .
        - DSGVO-compliant Telegram Bot operational alerts (low stock, Schwund alerts, object status changes).

7. Phased Execution Roadmap (Offline Localhost First -> GitHub Production Push)

Following **Chapter 1.2 (Three-Phase Modular Development Methodology)** in your blueprint:

1. **Phase 0: Foundation, Localhost Environment & Storage Setup**
    - Configure local PostgreSQL + Storage adapter (Local/MinIO + S3/R2 ready).
    - Initialize the full PostgreSQL schema (Chapter 4 tables + RBAC module permissions + Sickness/Tasks/Payslips/Expenses extensions + RLS + Audit Triggers).
    - Build `/backend/core` (JWT Access/Refresh auth, RBAC & Module Permission guards, RLS session context, Audit logger).
2. **Phase 1: Isolated Module Builds (Backend + Frontend)**

- **Module 1 (HR & Employee Operations):** Employee directory, Smart Check-in/Check-out, Working hours log, Holiday (`Urlaub`) & Sickness (`Krankmeldung`) workflows, Monthly Salary Documents (`Lohnabrechnung`), and Employee Tasks.
- **Module 2 (Project, Region & Object Management):** Region (PLZ) -> Project -> Building Object hierarchy (`Planned -> Hausbegehung -> In Progress -> Tested -> Completed`), media/video gallery per Object, daily employee assignment matrix, and `Vertretung` delegation.
- **Module 3 (Inventory & Event-Sourced Material Ledger):** Static Lagers & Vehicle Lagers, Materials catalog, `Buchen` (CheckOut, Return, Consume), dynamic `SUM(quantity)` stock calculation, and mandatory-note `Schwund` reconciliation.
- **Module 4 (Equipment & Fleet Tracking):** Vehicle fleet management, high-value tool tracking with exclusive assignment constraint (`employee` XOR `vehicle`), tool handover logs.
- **Module 5 (Centralized Interactive Master Calendar):** Unified calendar reading `assignments` , `leave_requests` , `sickness` , and `objects` with blueprint color-coding (Solid Yellow, Faded Yellow, Green, Blue).
- **Module 6 (Accounting, Expenses & Document Vault):** Expense submission & approval workflow, receipt/invoice media viewer, monthly payslip management, financial summary (without n8n runtime dependency).
- **Module 7 (Field Sales & Hardware Upselling):** Sales catalog, mobile checkout (CASH/CARD) with live vehicle stock validation, PostgreSQL trigger stock deduction, live admin sales feed, and monthly employee commission (`Provision`) calculator.

3. **Phase 2: Dual-Portal Integration (`app.bruenel.de` & `portal.bruenel.de`) & Testing**
   - Wire up the Management Portal (`app.bruenel.de`) with dynamic module visibility per user account and existing modules (Supplier Matrix, BI, Mail, Legal Vault).
   - Wire up the Employee Mobile PWA (`portal.bruenel.de`) for field workers and team leads.
   - Run end-to-end validation of RLS policies, triggers, and file/video uploads on `localhost` .
4. **Phase 3: n8n Documentation & Production Deployment**
   - Write the complete `N8N_AUTOMATION_GUIDE.md` specification for future automation setup.
   - Upon your approval of the local build, push to `https://github.com/bruenel/app.bruenel.de` using your provided GitHub token.